

Documentação do Projeto - Assistente de Voz (LPN)

1. Resumo do Projeto

- Nome: Assistente de Voz (LPN)
- Objetivo: Construir um assistente de voz local capaz de reconhecer emoções na fala, responder a comandos, carregar e gerenciar uma agenda e interagir via voz.
- Escopo: Detecção de emoção por áudio (modelo pré-treinado em `models/speech_emotion_recognition.hdf5``), reconhecimento de comandos, síntese de voz e módulos para agenda e respostas.

2. Estrutura do Repositório

- `app.py``: Ponto de entrada (pode expor interface web ou servir como runner).
- `assistente_voz.py``: Script para executar o assistente de voz em modo interativo.
- `README.md``: Instruções resumidas e contexto do projeto.
- `anotacao.txt``: Notas e anotações do desenvolvimento.
- `models/speech_emotion_recognition.hdf5``: Modelo Keras/TensorFlow para reconhecimento de emoções na fala.
- `modules/``
 - `__init__.py``
 - `assistente_virtual.py``: Lógica central do assistente (orquestração de módulos).
 - `carrega_agenda.py``: Funções de carregamento e manipulação da agenda.
 - `comandos_respostas.py``: Mapeamento entre comandos e respostas/ações.
 - `voz_assistente.py``: Captura de áudio e síntese (TTS) de respostas.
- `recordings/``: Exemplos de áudio e gravações de teste.

3. Funcionalidades (detalhadas)

- Captura de áudio e reconhecimento de comandos:
 - Capta áudio do microfone, aplica pré-processamento e detecta comandos configurados em `comandos_respostas.py``.
- Síntese de voz (TTS):
 - Converte texto em fala para responder ao usuário usando o módulo `voz_assistente.py``.
- Reconhecimento de emoções na fala:
 - Usa o modelo `models/speech_emotion_recognition.hdf5`` para classificar emoções (ex.: feliz, triste, neutro, raiva).
 - Pré-processamento esperado: extração de features (MFCCs, delta, etc.) — função de pré-processamento provável no módulo de assistente.
- Gerenciamento de agenda:

- Adicionar, listar e remover compromissos via `carrega\_agenda.py`.

- Respostas programadas e adaptativas:

- `comandos\_respostas.py` associa intenções a respostas ou ações (incluir respostas dinâmicas baseadas em contexto ou emoção detectada).

4. Instalação e Execução

Recomenda-se usar um ambiente virtual para isolar dependências.

Passos básicos:

1) Criar e ativar ambiente virtual:

```
python3 -m venv .venv  
source .venv/bin/activate
```

2) Instalar dependências (se existir `requirements.txt`):

```
pip install -r requirements.txt
```

Se não houver `requirements.txt`, instalar dependências comuns:

```
pip install numpy scipy librosa tensorflow keras pyaudio sounddevice pytsx3  
SpeechRecognition
```

3) Preparar dispositivo de áudio (microfone/alto-falante) e permissões.

4) Executar a aplicação:

```
python assistente_voz.py
```

ou, se houver interface web:

```
python app.py
```

Observações:

- Em sistemas Linux, a instalação de `pyaudio` pode requerer pacotes do sistema (`portaudio19-dev`).

- Ajustar nomes de bibliotecas conforme as usadas no projeto real.

5. Testes e Validação

- Testes manuais:
 - Usar arquivos em `recordings/` para testar reconhecimento de emoção e comandos.
 - Casos de uso: adicionar compromisso, perguntar agenda, solicitar estado emocional.
- Métricas para o modelo de emoção:
 - Acurácia, precisão, recall e F1 por classe.
 - Matriz de confusão para análise de erros.

6. Como Apresentar uma Versão da Aplicação (Release) e Entrega P2

Este item descreve como preparar uma versão pronta para demonstração e o que incluir na entrega P2 (segunda entrega/relatório).

- Preparar versão (release):
 1. Criar um branch de release: ``git checkout -b release/v1.0``.
 2. Atualizar ``README.md`` e este arquivo ``documentacao_projeto.txt`` com instruções de execução.
 3. Gerar ``requirements.txt``: ``pip freeze > requirements.txt`` após instalar dependências em ambiente limpo.
 4. Tagging: ``git tag -a v1.0 -m "Release v1.0 - P2"`` e ``git push origin v1.0``.
 5. Preparar amostras em ``recordings/`` que demonstrem casos relevantes (comando + emoção).
- Conteúdo da apresentação (demo):
 - Introdução rápida ao objetivo e arquitetura (1-2 slides).
 - Demo ao vivo: mostrar captura de comando, resposta TTS, e classificação de emoção.
 - Mostrar funcionamento da agenda (adicionar/listar).
 - Encerrar com limitações e próximos passos.
- Entrega P2 (recomendada):
 1. Relatório técnico (PDF) contendo:
 - Descrição do problema e objetivos.
 - Arquitetura do sistema (diagrama simples).
 - Descrição do dataset e pipeline de treinamento do modelo de emoção.
 - Resultados e métricas.
 - Testes e instruções de reprodução.
 - Conclusões e trabalho futuro.
 2. Código-fonte etiquetado (tag Git).
 3. ``requirements.txt`` e instruções de instalação.
 4. Vídeo curto demonstrando as funcionalidades principais.
 5. Exemplo de entrega empacotada (zip) com instruções para execução local.

7. Boas práticas para apresentações e P2

- Automatizar scripts de execução para facilitar a avaliação.
- Incluir testes automatizados mínimos (unitários) para funções críticas.
- Documentar formatos de entrada/saída (por exemplo, formato de arquivos de agenda, parâmetros esperados para o modelo).
- Garantir que amostras de áudio estejam incluídas e sejam pequenas para troca rápida.

8. Referências

- Bibliotecas e ferramentas úteis:
 - TensorFlow/Keras — <https://www.tensorflow.org/>
 - Librosa — <https://librosa.org/>
 - PyAudio — <https://people.csail.mit.edu/hubert/pyaudio/>
 - SpeechRecognition — <https://pypi.org/project/SpeechRecognition/>
 - pyttsx3 — <https://pypi.org/project/pyttsx3/>
- Artigos e leituras sobre reconhecimento de emoção na fala (exemplos):
 - Schuller, B., Batliner, A. — Computational Paralinguistics: Emotion, Affect and Personality in Speech and Language Processing.
 - Eyben, F., et al. — OpenSMILE: The Munich Versatile and Fast Open-Source Audio Feature Extractor.

9. Pontos para desenvolvimento futuro

- Melhorar robustez do reconhecimento de fala em ambientes ruidosos.
- Re-treinar modelo de emoção com dados balanceados e maiores samples.
- Implementar um serviço de API com endpoints REST para integração com outras aplicações.
- Adicionar persistência robusta para a agenda (banco de dados leve: SQLite).
- Internacionalização e suporte a múltiplos idiomas.

Contato e autores

- Repositório atual: estrutura local em ``/home/avrovulcan/Documents/LPN``.
- Para dúvidas ou contribuições, abrir issue ou PR no repositório Git associado.

-- FIM --